

Digital Matrices: Building Instagram-Style Filters

AIAA3201: Introduction to Computer Vision – Lab 01

Welcome to the machine perspective. Today, we move from being users of technology to creators of it. We will demystify the “magic” of AI perception by building a filter engine from scratch.

The Mission: From Theory to Implementation



1. Environment Setup

Configuring the "Darkroom" (`Python & OpenCV`).



2. The Core Concept

Understanding `'Image Processing = Numerical Operations'`.



3. The Blueprint

Deconstructing the math behind 5 classic filters.



4. Execution

Generating a personalized photo montage.

The Promise: By the end of this session, you will not just understand how filters work—you will have the code to build your own.

How AI Sees the World

You: Beach 🏖️



You: Beach 🏖️

Computer Vision

Computer: [123, 45, 67...]

[123]	[123]	[45]	[45]	[67]	[67]
[123]	[123]	[45]	[45]	[67]	[67]
[45]	[45]	[123]	[123]	[45]	[45]
[45]	[45]	[67]	[67]	[123]	[123]
[67]	[67]	[45]	[45]	[123]	[123]

Computer: [123, 45, 67...]

Digital Anatomy:

Images are not pictures; they are digital matrices. Every pixel is a coordinate containing three channels: Red, Green, and Blue (RGB).

The Formula:

Applying a filter is simply performing mathematical operations on these numbers.

```
New_Pixel = Old_Pixel * Weight + Bias
```

The Target: 5 Classic Filter Effects in Inter



Vintage Warm

Film texture simulations.

Cool Tone

Atmospheric temperature shifts.

High Contrast

Enhanced visual impact.

Faded

Nostalgic, low-saturation look.

Grayscale

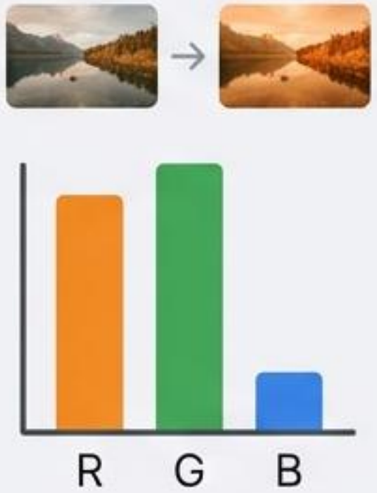
Classic B&W conversion.

This visual output is the direct result of the code we are about to write.

Logic Block A: Manipulating Color Temperature

Painting with Numbers

Vintage Warm



Multiply Red and Green channels by a weight > 1.0 to ‘warm’ the image.

Cool Tone

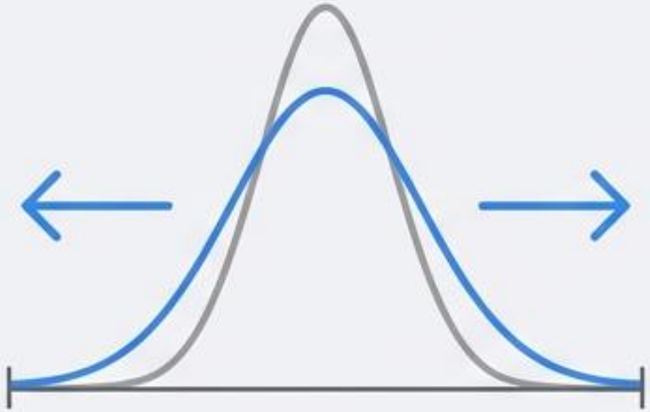


Enhance the Blue (B) channel while reducing the Red (R) and Green (G) channels.

```
Code Concept: image[:, :, channel] * weight
```

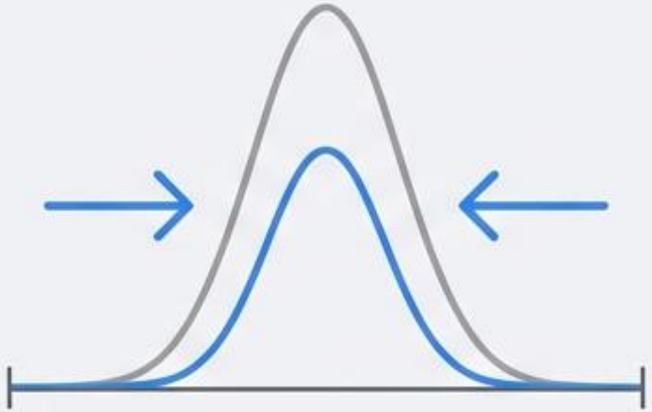
Logic Block B: Controlling Light & Dynamic Range

High Contrast



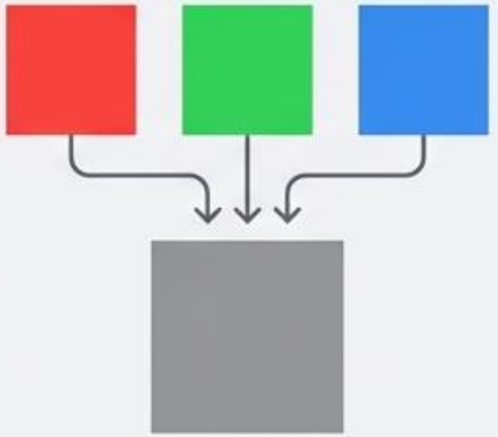
Histogram Stretching: Utilizing the full dynamic range. Darks become darker, lights become lighter.

Faded



Compression: Reducing the dynamic range and saturation to mimic aged photos.

Grayscale



Space Conversion: Converting 3-channel RGB space into single-channel Grayscale.

The Toolkit: Technical Requirements

Tech Stack



Python

Version 3.8 or higher

OpenCV

```
opencv-python
```

The heavy lifter for image processing tasks.

NumPy

The math library for matrix operations.

Matplotlib

Used for rendering and displaying results.

OS Compatibility: Windows / macOS / Linux.

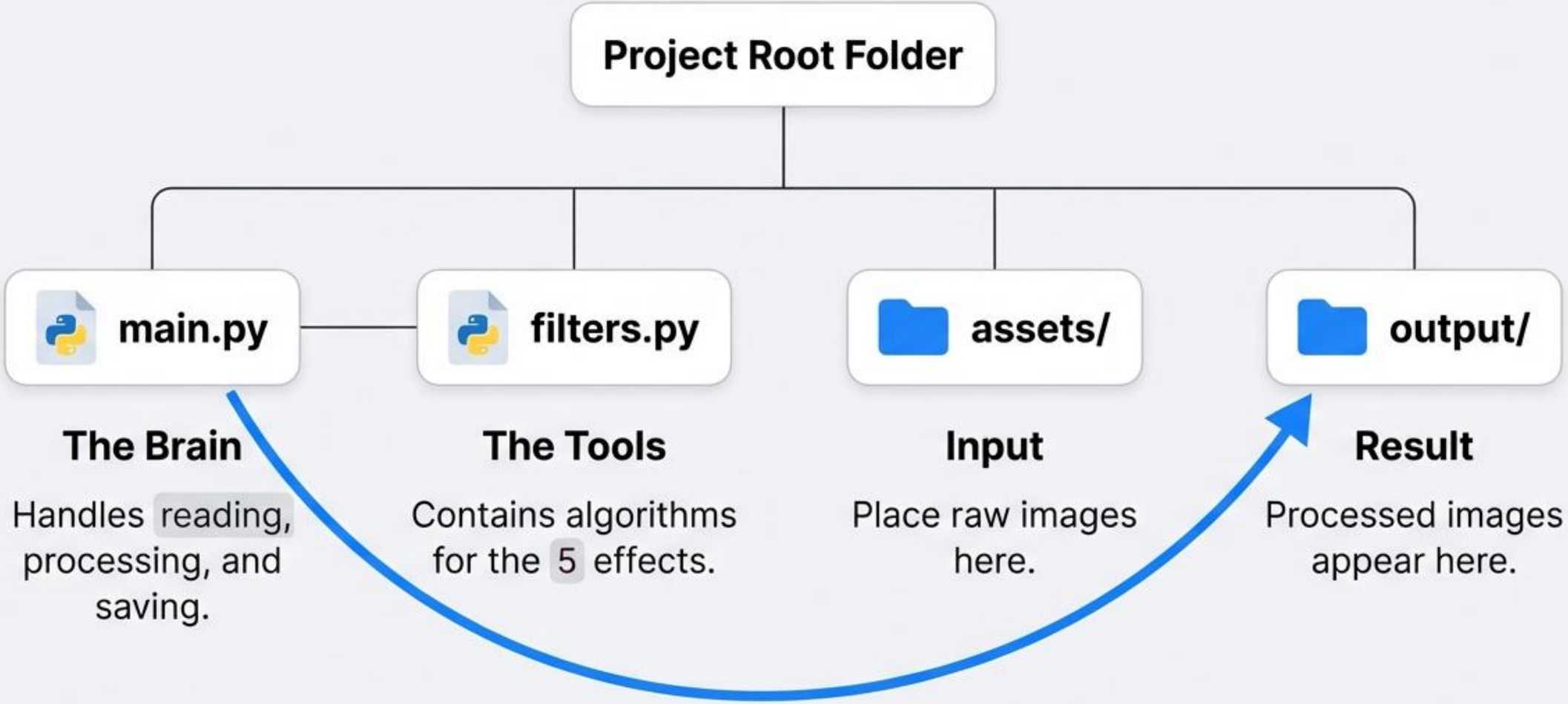
Phase 1: Environment Setup

```
$ python --version
# Ensure v3.8+
$ pip install opencv-python numpy matplotlib
[...installing progress bars...]
$ python
>>> import cv2
```

Alert

macOS Note: If your system defaults to Python 2.7, you must use 'python3' and 'pip3'.

Phase 2: Project Architecture



Code Spotlight: Inside filters.py

filters.py

```python
def apply_vintage_warm(image):
    # Implementation...

def apply_cool_tone(image):
    # Implementation...

def apply_high_contrast(image):
    # Implementation...

def apply_faded(image):
    # Implementation...

def apply_grayscale(image):
    # Implementation...
```

apply_vintage_warm
Weighted channel adjustment.

apply_cool_tone
Enhance Blue channel logic.

apply_high_contrast
Histogram manipulation.

apply_faded
Saturation reduction.

apply_grayscale
cv2.cvtColor conversion.

The Engine: Inside main.py

Read

Scan `assets/` folder for images.

Process

Loop through images & apply all 5 filters from `filters.py`.

Montage

Stitch **Original + 5 Filters** into a **2x3 grid**.

Save

Write files to `output/` folder.

```
create_filter_montage(original, results)
```

Phase 3: Run the Lab

1 Download Project

Ensure a code snippet like `main.py`, `filters.py`, and `folders` are in the same directory.

2 Prepare Images

Place your own personal photos into the `assets/` folder.

3 Run Program

Execute command: `python main.py` (or `python3 main.py`).

4 View Results

Open the `output/` folder to see your generated filter montages.

Troubleshooting & Support

Installation Fails

Issue: pip timeout or error

Solution: Use a domestic mirror source or try installing `'opencv-python-headless'` snippet in JetBrains

File Not Found

Issue: Python can't find image

Solution: Check `'assets/'` `JetBrains` folder. Use forward slashes (/) in paths, not backslashes (\).

No Display

Issue: Window flashes or doesn't open

Solution: The code saves to disk by default. Open the files in your OS viewer manually.

Deliverables & Next Steps

✅ Acceptance Criteria

- ✅ **Environment Success:** Python & OpenCV installed.
- ✅ **Code Execution:** Generated at least one filter montage.
- ✅ **Concept Check:** Explain that “Images are matrices”.

⚙ Advanced Challenges (Optional)

1. Modify numeric weights to create a signature look.
2. Implement real-time webcam processing.

Computer Vision is numbers → understanding.
You have now taken the first step.